

PVT

MT8676 Hypervisor Suspend & Resume

Common Flow

版本: v1
出版日期: 2026/1/7

版本记录

版本	日期	作者	描述
v1	2026/1/7	杨娟陶	正式版

目录

1.1 概述.....	4
1.2 架构/流程概要.....	5
1.2.1 Hypervisor Suspend 流程.....	5
1.2.2 Hypervisor Resume 流程.....	6
1.3 唤醒源介绍.....	8
1.4 STR 问题分析.....	9
1.4.1 STR 触发方式.....	9
1.4.2 STR 常用 debug 命令.....	10
1.4.3 关键 log 查找.....	11
1.4.4 关键 log 介绍.....	11
1.4.5 典型问题分析	13

1.1 概述

本章节主要介绍 MT8676 Hypervisor Suspend/Resume 流程。

在 MT8676 平台上，Suspend 即 Suspend to RAM，此状态使所有的设备进入低功耗状态，仅保留 RAM 自刷新。

Suspend Power State:

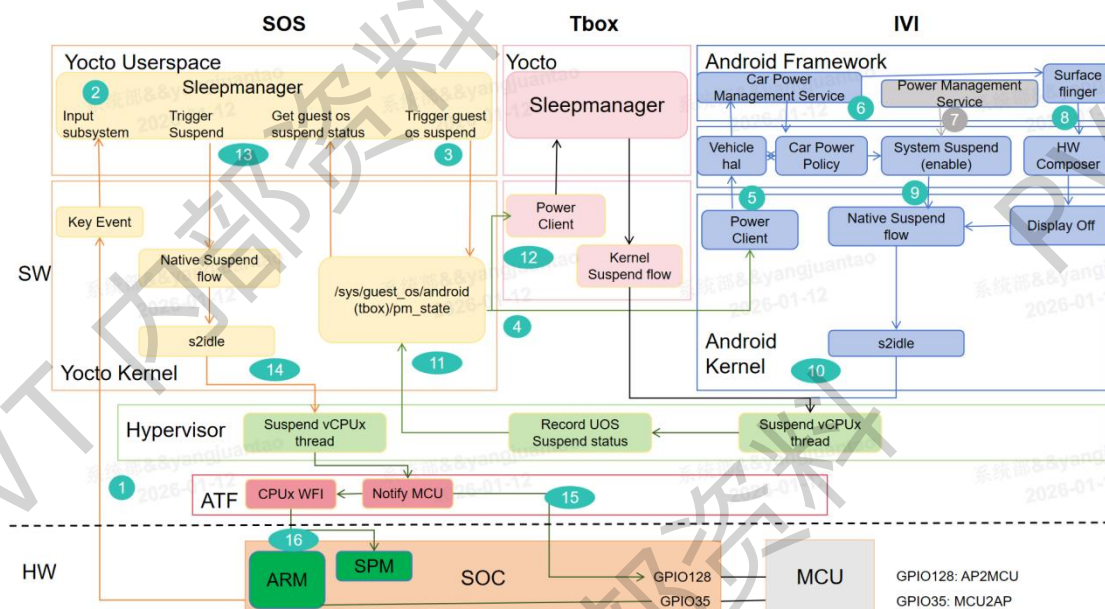
- Tasks “freezed”
- 外部设备/内部部分模块关电或者进入低功耗模式
- System PLL/clock close
- ARM off
- DRAM 自刷新
- PMIC 进入低功耗模式
- VCORE off
- SPM run
- 等待硬件唤醒事件

在 MT8676 Hypervisor (L+L+A) 系统上，Host OS SoS 与 Guest OS Tbox 系统都是 Yocto，Guest OS IVI 系统是 Android。系统级 Suspend 由 Host OS 发起，触发 Guest OS 运行 Suspend Flow，Guest OS 都完成 Suspend Flow 之后，Host OS 可以进入 Suspend，Host OS Suspend 完成之后整个系统才能进入省电的深度休眠模式。

1.2 架构/流程概要

1.2.1 Hypervisor Suspend 流程

Hypervisor Suspend 框架图如下 (以 EINT 触发系统进出休眠为例)：



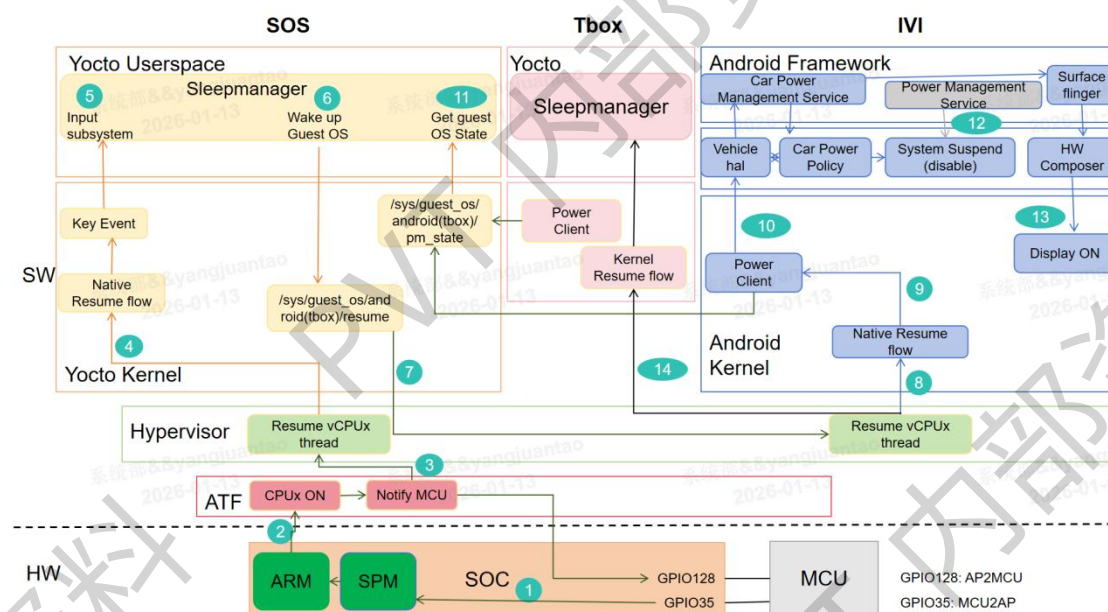
主要的流程说明:

1. 平台进出休眠主要由 MCU 发起, MCU 通过 GPIO35 (MCU2AP) 的状态控制 SoC 进出休眠, GPIO35 被拉低后,会触发中断,该中断会被 Hypervisor 分发给 SoS 系统处理,SoS Kernel 收到中断后,将 Key Event 上报。
2. 在 SoS UserSpace 中,有一段控制逻辑 SleepManager,用于管理 Guest OS 的电源状态,这层逻辑主要是 SoC 通过 Input Sub System 获取 Key Event 来触发的。
3. 当检测到休眠事件后 SoS UserSpace 写 powerkey 到 /sys/guest_os/android/pm_state, 向 android 发起休眠请求。
4. SoS 的 Kernel Driver 通过 Mailbox 的方式将 Suspend 请求通知到 Android 的 power client。
5. 在 power client 中,上报虚拟 Power Key (Keyevent=87), 触发 Guest OS 端完整的休眠流程。
6. Guest OS IVI 由于 Vehicle HAL 没有现成的接口对接 power client,所以这里增加 Input sub system 支持,等待 Kernel 端送来的虚拟 Power Key,VHAL 收到后会通知 Car Power Management Service 启动休眠流程。
7. Power Management Service 依然监听 Power Key Event(Keyevent=116), 这里没有执行 PMS Flow。
8. 触发关闭显示相关硬件。
9. CPMS 在没有唤醒锁检查的情况下触发 Suspend Flow, 触发 Android Kernel Suspend。Kernel Suspend Flow 与原生系统中的流程相同。根据执行顺序,它依次调用设备注册 Prepare/Suspend/Suspend_late 回调函数,在休眠阶段为每个 Device driver 完成必要的准备工作。
10. 各 Device 休眠后,送 s2idle SMC 指令到 ATF 去关闭 CPU core,指令会被 Hypervisor 拦截,并挂起对应的 vCPU 线程。Android 的休眠完成, Hypervisor 会记录的 Android 状态将从 running 改为 suspend,方便 Host OS 通过节点查询。

11. Host OS 端触发 Guest OS 端休眠后会阻塞并等待，直到从节点/sys/guest_os/android/pm_state 获取到 android 为 Suspend 的状态。
12. 当检测到安卓休眠事件后 SoS UserSpace 写 powerkey 到/sys/guest_os/tbox/pm_state，向 tbox 发起休眠请求并监听 tbox 状态。
13. 待 Guest OS 都 Suspend 后，Host OS 端上报 key code 116 触发关闭显示相关，然后再检查 Wakelock 状态，如无锁，则触发 Host OS Kernel Suspend Flow (echo mem > /sys/power/state)。
14. 执行 Host OS Kernel Suspend，完成 Device Suspend 后，送 s2idle SMC 指令到 ATF 去关闭 CPU core，指令会被 Hypervisor 拦截，并挂起对应的 vCPU 线程，然后虚拟机做好 backup，Hypervisor 将 s2idle 指令再发送给 ATF，将物理 CPU 真正关闭。
15. 在 ATF 中，CPU0 执行 WFI 指令前，先将 GPIO128 (AP2MCU)拉低，通知 MCU，SoC 端已经完成 Host OS 和 Guest OS 的 Suspend Flow。
16. 待 Arm 相关的硬件被关闭后，SPM (System Power Manage，用于电源管理的 MCU) 将接管整个系统的资源，控制 DRAM 进入自刷新模式，关闭 26M 和 VCORE，通知 PMIC 进入 Low Power Mode。至此，整个休眠流程完成，系统处于低功耗状态，等待唤醒事件发生。

1.2.2 Hypervisor Resume 流程

Resume Flow 与 Suspend 相反，可以参考下图：



主要流程：

1. MCU 收到唤醒事件后，拉高 GPIO35 (MCU2AP)，SPM 收到 EINT 后，依次打开释放的资源，如通知 PMIC 离开 Low Power Mode，打开 VCORE 和 26M，发送命令通知 DRAM 离开自刷新。
2. 给 CPU0 上电，执行 CPU Resume 流程，再跳到 Suspend 前设定的地址，开始执行系统 Resume 流程。
3. 拉高 GPIO128，将 SoC 已经醒来的消息发送给 MCU。(可根据需求调整通知 MCU 的时间点，如等 Android Resume 完成再拉高 GPIO128)。
4. Hypervisor 中 vCPUx 线程恢复后，跳到 Host OS Linux Kernel，再执行 Native Kernel Resume Flow，将唤醒事件通过 Key Event 上报。

5. SoS 的 SleepManager 通过 Input Subsystem 上报的事件判断后, 将决定再次休眠或者继续唤醒 Flow, 像 MCU 拉高 GPIO35 这种就会继续唤醒 Flow。
6. 如果是可以唤醒整个系统的 Key Event, 则上报 keyevent 116 去 resume Weston, 并 hold wakelock, 防止再次休眠。
7. 写节点/sys/guest_os/android/resume, 触发安卓唤醒。
8. Hypervisor 收到后, 恢复 vCPU0 线程。
9. Enable Non-Boot CPUs, 恢复 vCPU1~7 线程, 开始 IVI Kernel Resume Flow。
10. IVI 基于 CPMS 架构, 无需判断 keyevent, 默认所有事件都可以触发整个系统 Resume。
11. 在 Host OS 端, 可以通过节点/sys/guest_os/android/pm_state, 使用 Mailbox 机制查询 Android 状态, 如果 Guest OS 端 kernel Resume 完成则更新状态为 Running。
12. Android 端, Car Power Management Service 收到唤醒事件后, Disable System Suspend, 防止误触发休眠。
13. 通知打开显示相关模块。
14. sos 监听到安卓唤醒结束后, sos 写节点/sys/guest_os/tbox/resume 唤醒 Tbox 端, 由 SleepManager 主动 hold wakelock。

1.3 唤醒源介绍

目前 MT8676 支持的唤醒源列表如下：

Name	Control Bit	Description	Can it be Disabled
R12_PCM_TIMER	0	设定计时器，超时后唤醒系统	Y
R12_KP_IRQ_B	2	检测到键盘按下/释放动作	Y
R12_APWDT_EVENT_B	3	RGU 在 suspend mode 中唤醒 CPU	Y
R12_APXGPT1_EVENT_B	4	AP GPT 计时器超时事件	Y
R12_CONN2AP_SPM_WAKEUP_B	5	Connectivity IC (Wi-Fi/BT/GPS) 唤醒系统	Y
R12_EINT_EVENT_B	6	EINT 事件唤醒系统	N
R12_CONN_WDT_IRQ_B	7	Connectivity IC watchdog timeout 唤醒系统	Y
R12_CCIF0_EVENT_B	8	MD to AP CCIF 唤醒事件	Y
R12_CCIF1_EVENT_B	9	MD to AP CCIF 唤醒事件	Y
R12_SSPM2SPM_WAKEUP_B	10	SSPM 相关唤醒系统	Y
R12_SCP2SPM_WAKEUP_B	11	SCP sensor 相关唤醒系统	N
R12_ADSP2SPM_WAKEUP_B	12	ADSP 相关唤醒系统	Y
R12_USBX_CDSC_B	14	USB 相关唤醒系统	Y
R12_USBX_POWERDOWN_B	15	USB 远程唤醒	Y
R12_SYS_TIMER_EVENT_B	18	System timer 唤醒系统	N
R12_EINT_EVENT_SECURE_B	19	EINT event 唤醒系统	N
R12_SCP_CIRQ_IRQ_B	22	SCP_CIRQ 唤醒事件	Y
R12_MD2AP_PEER_EVENT_B	23	MD 相关唤醒系统	Y
R12_MD1_WDT_B	25	MD1 看门狗超时	Y
R12_REG_CPU_WAKEUP_B	28	内部唤醒源	N
R12_APUSYS_WAKE_HOST_B	29	APUSYS 的唤醒源	Y
R12_PCIE_WAKEUP_EVENT_B	30	PCIe 相关唤醒源	Y
R12_MSDC_WAKEUP_EVENT_B	31	MSDC 相关唤醒源	Y

您可以按照以下方式禁用列表中可以禁用的唤醒源。但是，请注意，只有在“Can it be Disabled”列中标注为可禁用（Y）的唤醒源才能被禁用：

使用系统提供的调试命令可以快速禁用特定的唤醒源：

1. 使用如下命令读取当前系统唤醒源设定：

```
cat /proc/mtk_lpm/power/suspend_ctrl | grep -i wake_src
```

2. 在上表中的“Control Bit”栏位中找到您想要禁用唤醒源所对应的 control bit 位，然后将读出的 wake_src 的对应 bit 写 0，最后用下述命令将修改后的 wake_src 写进系统（不需要禁用的唤醒源的 control bit 位不进行修改）：

```
echo wake_src 0x***** > /proc/mtk_lpm/power/suspend_ctrl
```

3. 如果想要再启用某个唤醒源，就将对应的 control bit 再写 1 即可。

1.4 STR 问题分析

1.4.1 STR 触发方式

整机休眠: `echo stress > /sys/kernel/powerkey/stress`

休眠安卓: `echo powerkey > /sys/guest_os/android/pm_state`

休眠 tbox: `echo powerkey > /sys/guest_os/tbox/pm_state`

rtc 唤醒, 30 是指 30s: `echo +30 > /sys/class/rtc/rtc0/wakealarm`

单独唤醒安卓: `echo 0 > /sys/guest_os/android/resume`

持锁唤醒 tbox: `echo 1 1 > /sys/guest_os/tbox/resume`

不持锁唤醒 tbox: `echo 1 2 > /sys/guest_os/tbox/resume`

看安卓状态: `cat /sys/guest_os/android/pm_state`

看 tbox 状态: `cat /sys/guest_os/tbox/pm_state`

客制化(acc)

1.4.2 STR 常用 debug 命令

//休眠时仍然尝试打印日志, 会增加功耗, 仅 debug 用

```
echo 0 > /sys/module/printk/parameters/console_suspend
```

//打印更多 log 信息

```
echo 7 > /proc/sys/kernel/printk
```

//打开 kernel log

```
echo 1 > /proc/mtprintk
```

//查看 block by/wake up by, 26M_off_pct 是 0 的时候才有 blocked by 打印, 不是 0, 说明最终 26M 有被关, 需要时间

sos 下这两个命令, 看下 config 是不是开的

```
zcat /proc/config.gz | grep CONFIG_PM_DEBUG
```

```
zcat /proc/config.gz | grep CONFIG_PM_SLEEP_DEBUG
```

然后 sos 端下

```
echo 1 > /sys/power/pm_debug_messages
```

//如果软件已经进入 low power mode, 但是功耗未达标, 需要接着确定硬件, 休眠前 dump gpio 状态

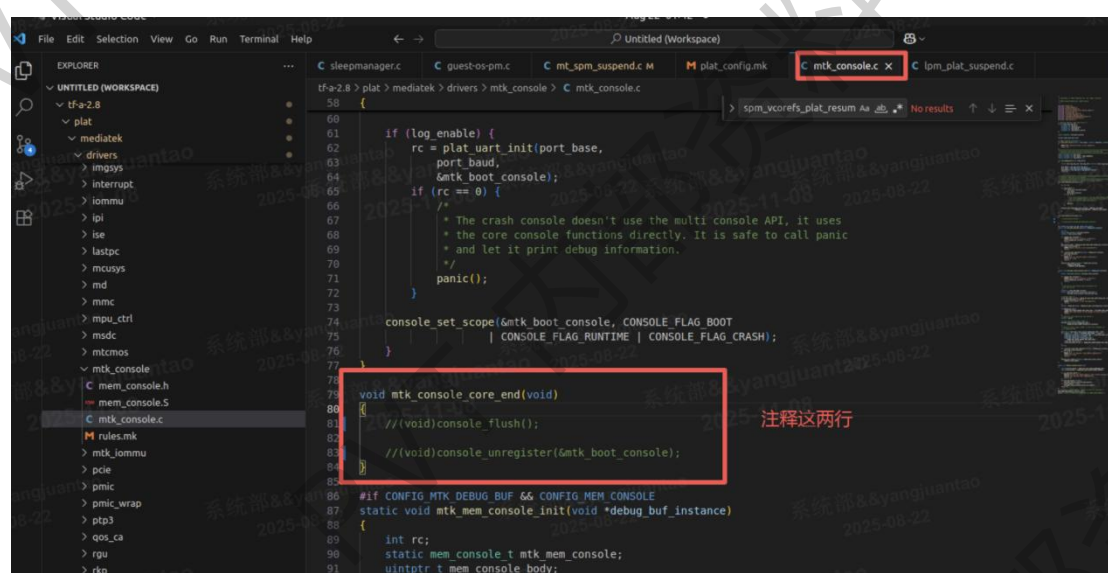
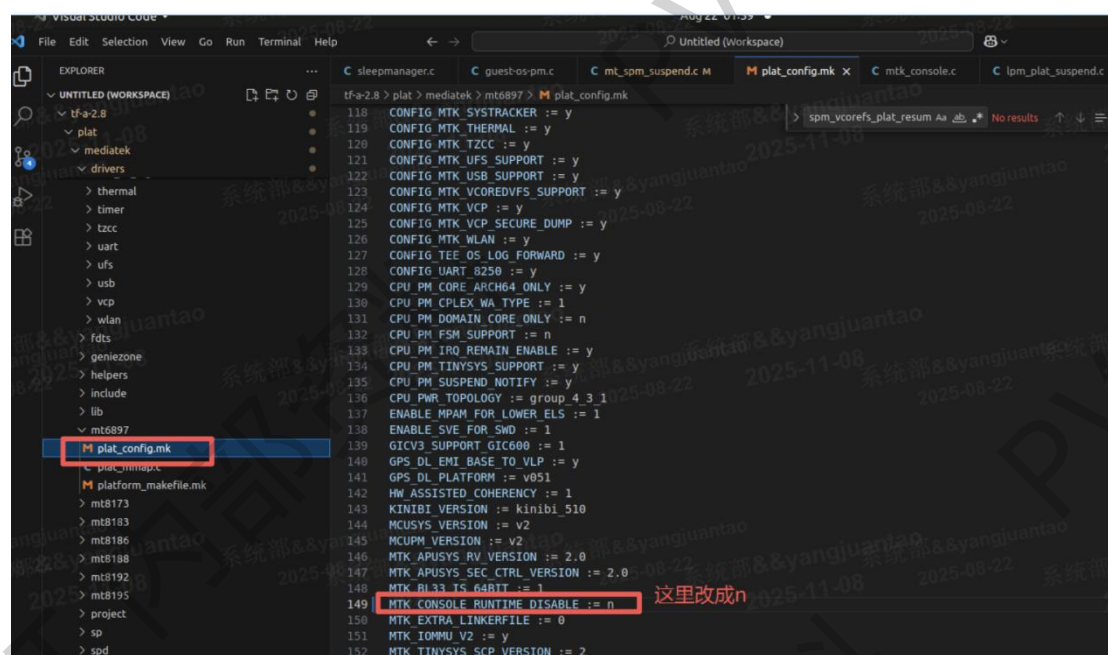
```
echo gpio_dump 1 > /proc/mtk_lpm/suspend/suspend_state
```

```
<5>[ 1714.940614][T10300002393] sleepmanager: PIN: (MODE) (DIR) (DOUT) (DIN) (DRIVE) (SMT) (IES) (PULL_EN) (PULL_SEL) (R1 R0)
<5>[ 1714.942155][T10300002393] sleepmanager: 000: 0001001110
<5>[ 1714.943132][T10300002393] sleepmanager: 001: 0111001110
<5>[ 1714.944065][T10300002393] sleepmanager: 002: 0000001110
```

//查看 kernel 中有哪些模块持锁

```
cat /sys/kernel/debug/wakeup_sources
```

//打开 atf log



1.4.3 关键 log 查找

拿到文件后, 先和报案人确认问题时间点, 然后在 sos kernel log 贴下面一大串开始搜索
suspended for|pm: suspend|suspend wake up by |suspend enter|PM: Pending Wakeup
Sources|Wakeup Source|Restarting tasks|Freezing user space processes|Syncing
filesystem|complete after|aborted after|failed after|Resume caused by |is
pending|eint_mask|wakeuper by|Resume cause|pm_state|wake up by|read tc time|set al time|sh:

1.4.4 关键 log 介绍

kernel log:

//开始休眠安卓

sh: pm_state_store:powerkey

//开始休眠 tbox

sh: tbox_pm_state_store:powerkey

//检测 log, vm0(安卓), vm1(tbox)

host get vm0 pm_state 1

pm_state 状态如下:

#define GUEST_PM_SYSTEM_OFF	0
#define GUEST_PM_RUNNING	1
#define GUEST_PM_SUSPEND_TO_RAM	2
#define GUEST_PM_SUSPEND_TO_IDLE	3

//关闭 usb

sh: mtu3 16721000.usb2: ssusb_set_mode none

//进入休眠时间

sleepmanager: [name:spm&] PM: suspend entry 2025-09-17 15:36:54.497349 UTC;android time 2025-09-17 15:36:54.497349

//kernel 休眠结束

swapper/3: [name:spm&][lpm_suspend_s2idle_prompt:124] - suspend enter

//唤醒后会打印出唤醒源, 如果 debug_flag 后两位不是 ff, 代表休眠前有子系统起来工作, 26M_off_pct 代表 26M 休下去的占比, 如果是 0 说明没有深休

swapper/3: [name:spm&][SPM] suspend wake up by R12_EINT_EVENT_B, timer_out = 19316402, r13 = 0x1200, debug_flag = 0x1003f3ff 0x207e071, r12 = 0x40, r12_ext = 0x40, raw_sta = 0x40 0x0 0x0, idle_sta = 0x0, req_sta = 0x100000 0x0 0x0 0x0 | 0x0 0x380000 0x0 0x78 | 0x0 0x0 0x1fcd800 0x0 | 0x0 0x0 0x0 0x0 | 0x40, spm_src_req = 0x8f20000, cg_check_sta = 0x0, isr = 0x0, sw_rsv = 0x0 0x0 0x0 0x80000000 | 0x126bd57 0xffffffff 0x0, raw_ext_sta = 0x84cb255, wake_misc = 0x180030, sw_flag = 0x8a0014 0x50000, b_sw_flag = 0x8a0014 0x50000, clk_settle = 0x60fe, wlk_cntcv_l = 0x94699ab0, wlk_cntcv_h = 0x2, 26M_off_pct = 99, vcore_off_pct = 99

//确定休眠时间 (要确定约等于退出时间-开始时间, 不然就是假休眠)

swapper/3: [name:spm&][SPM] suspended for 589.489 seconds

//退出休眠时间

sleepmanager: [name:spm&] PM: suspend exit 2025-09-17 15:46:45.161016 UTC;android time 2025-09-17 15:46:45.161016

//rtc 相关

read tc time //读出当前时间

set al time //设置 rtc 时间

1.4.5 典型问题分析

1.4.5.1 休眠问题

A) Userspace freeze 失败

log 表现为:

```
<3>[ 852.415255][T1102726] sleep_manager: [name:process]&Freezing of tasks failed after 20.010 seconds (1 tasks refusing to freeze, wq_busy=0):
<6>[ 852.417092][T1102726] sleep_manager: [name:oom kill]&OOM killer enabled.
<6>[ 852.417959][T1102726] sleep_manager: [name:process]&Restarting tasks ...
```

1.确认这俩 config 有开

zcat /proc/config.gz | grep CONFIG_PM_DEBUG

zcat /proc/config.gz | grep CONFIG_PM_SLEEP_DEBUG

2. 输入下面 command

echo 1 > /sys/power/pm_debug_messages

3. 重新复现

4.哪个 task 拒绝 freeze, 就可以打印出来

```
80710 [20:29:16:008] 221.913438[T10000002426] sleepmanager: Freezing user space processes failed after 20.007 seconds (1 tasks refusing to freeze, wq_busy=0):
80711 [20:29:16:008] 221.915124[T10000002426] sleepmanager: taskmdprobe state:D stack:0 pid:2405 ppid:2396 flags:0x00000209
80712 [20:29:16:008] 221.916978[T10000002426] sleepmanager: Call trace:
80713 [20:29:16:008] 221.917204[T10000002426] sleepmanager: _switch_to+0x180/0x344
80714 [20:29:16:008] 221.918050[T10000002426] sleepmanager: _schedule+0x5b8/0x990
80715 [20:29:16:009] 221.918762[T10000002426] sleepmanager: schedule+0x78/0x80
80716 [20:29:16:009] 221.919513[T10000002426] sleepmanager: schedule_timeout+0xac/0x164
80717 [20:29:16:009] 221.920329[T10000002426] sleepmanager: do_wait_for_completion+0x108/0x1b0
80718 [20:29:16:009] 221.921179[T10000002426] sleepmanager: wait_for_completion_timeout+0x48/0x68
80719 [20:29:16:009] 221.922106[T10000002426] sleepmanager: mtk_i2c_do_transfer+0x1fc/0x1b2c [g59xx touch 140000003000000474e550094cbdf11879670]
80720 [20:29:16:010] 221.923630[T10000002426] sleepmanager: mtk_i2c_transfer+0x258/0x40c [i2c_devxx 140000003000000474e550094cbdf11879670]
80721 [20:29:16:010] 221.925069[T10000002426] sleepmanager: _i2c_transfer+0x26c/0x7f8
80722 [20:29:16:010] 221.925873[T10000002426] sleepmanager: i2c_transfer+0xc8/0x13c
80723 [20:29:16:010] 221.926640[T10000002426] sleepmanager: gtp_i2c_read+0x18/0x32 [g59xx touch 140000003000000474e550094cbdf11879670]
80724 [20:29:16:011] 221.928143[T10000002426] sleepmanager: i2c_device_probe+0x1e/0x20 [g59xx touch 140000003000000474e550094cbdf11879670]
80725 [20:29:16:011] 221.929707[T10000002426] sleepmanager: gtp_probe+0x608/0x6d4 [g59xx touch 140000003000000474e550094cbdf11879670]
80726 [20:29:16:011] 221.931126[T10000002426] sleepmanager: i2c_device_probe+0x1e/0x20
80727 [20:29:16:011] 221.931994[T10000002426] sleepmanager: really_probe+0x114/0x48c
80728 [20:29:16:011] 221.932744[T10000002426] sleepmanager: _driver_probe_device+0xa3/0x1e5
80729 [20:29:16:012] 221.933654[T10000002426] sleepmanager: driver_probe_device+0x3e/0x1a5
80730 [20:29:16:012] 221.934498[T10000002426] sleepmanager: _driver_attach+0xbe/0x1e4
80731 [20:29:16:012] 221.935308[T10000002426] sleepmanager: bus_for_each_dev+0x104/0x160
80732 [20:29:16:012] 221.936162[T10000002426] sleepmanager: driver_attach+0x24/0x34
80733 [20:29:16:012] 221.936936[T10000002426] sleepmanager: bus_add_driver+0x14c/0x294
80734 [20:29:16:013] 221.937742[T10000002426] sleepmanager: driver_register+0x65/0x108
80735 [20:29:16:013] 221.938549[T10000002426] sleepmanager: i2c_register_driver+0x10/0x30
80736 [20:29:16:013] 221.939376[T10000002426] sleepmanager: init_module+0x2c/0x8fc [g59xx touch 140000003000000474e550094cbdf11879670]
80737 [20:29:16:013] 221.940746[T10000002426] sleepmanager: do_init_module+0x5/0x15f
80738 [20:29:16:013] 221.941537[T10000002426] sleepmanager: do_init_module+0x58/0x235
80739 [20:29:16:014] 221.942322[T10000002426] sleepmanager: load_module+0x10b/0x1214
80740 [20:29:16:014] 221.943103[T10000002426] sleepmanager: _arm64_sys_init_module+0x22c/0x330
80741 [20:29:16:014] 221.944015[T10000002426] sleepmanager: invoke_syscall+0x57/0x114
80742 [20:29:16:014] 221.944800[T10000002426] sleepmanager: _do_syscall_common+0x89/0x14c
80743 [20:29:16:014] 221.945570[T10000002426] sleepmanager: do_syscall_common+0x28
80744 [20:29:16:015] 221.946298[T10000002426] sleepmanager: el0_svc+0x35/0x68
80745 [20:29:16:015] 221.946992[T10000002426] sleepmanager: el0t_64_sync_handler+0x70/0xbe
80746 [20:29:16:015] 221.947837[T10000002426] sleepmanager: el0t_64_sync+0x1aa/0x1ac
80747 [20:29:16:015] 221.948747[T10000002426] sleepmanager: OOM killer enabled.
80748 [20:29:16:015] 221.949473[T10000002426] sleepmanager: Restarting tasks ...
80749 [20:29:16:016] 221.950272[T10500000074] kworker/u1:3 [name:spmk][SPM] mcuaps was not off
80750 [20:29:16:016] 221.952460[T10000002426] done.
```

B) 检查有 wakeup source

[20250926_09:05:41796][377.752848]nbl_vm_srv[2971]: [vmid:1][341.521844][T00000001648]

sleepmanager: Abort: Pending Wakeup Sources: ttyCMIPIC1

C) 休眠被异常唤醒

suspend wake up by R12_CCIF0_EVENT_B

正常唤醒也会打出这句 log, 要结合场景看属不属于异常唤醒源, R12_CCIF0_EVENT_B 属于 modem.

1.4.5.2 可以休眠但是电流不达标

A) 有 clock calltrace 打印

休眠时候会关 pll, 没关的会打出 suspend warning calltrace, suspend 会开始 retry, retry 三次后还没关,

则直接进入休眠。

```
[20251107_17:55:43819][ 273.248920][T10000002650] sleepmanager: sgmi-up soc:pdagmi-0-top: sgmi suspend noirq done
[20251107_17:55:43819][ 273.250451][T10000002650] sleepmanager: suspend warning(0m: mmp112 is on
[20251107_17:55:43819][ 273.251304][T10000002650] sleepmanager: suspend warning(0m: tvdp111 is on
[20251107_17:55:43820][ 273.252163][T10000002650] sleepmanager: suspend warning(0m: tvdp112 is on
[20251107_17:55:43820][ 273.253175][T10000002650] sleepmanager: [ck2_cousys_sel : prepared, 1, 0, 832000000, ck2_univp112_d3]
[20251107_17:55:43820][ 273.254551][T10000002650] sleepmanager: [ck2_disp_sel : prepared, 1, 0, 687499969, ck2_mmp112_d4]
[20251107_17:55:43820][ 273.255912][T10000002650] sleepmanager: [ck2_univp112_d3 : prepared, 1, 0, 832000000, univp112]
[20251107_17:55:43822][ 273.257267][T10000002650] sleepmanager: [ck2_mmp112_d4 : prepared, 1, 0, 687499969, mmp112]
[20251107_17:55:43822][ 273.258649][T10000002650] sleepmanager: [ck2_disp_ck : prepared, 1, 0, 687499969, ck2_disp_sel]
[20251107_17:55:43822][ 273.259999][T10000002650] sleepmanager: [mmp112 : enabled, 0, 1, 274999978, ck26m]
[20251107_17:55:43823][ 273.261348][T10000002650] sleepmanager: [tvdp111 : enabled, 0, 1, 247458817, ck26m]
[20251107_17:55:43829][ 273.262701][T10000002650] sleepmanager: [tvdp112 : enabled, 0, 1, 181359978, ck26m]
[20251107_17:55:43832][ 273.265368][T10000002650] sleepmanager: is_suspend_retry_stop: suspend cnt: 2
```

B) 无法进入 low power mode

若是直接打印出阻挡的模块, 直接找对应模块

```
[ 3147.743642][T103000000000] swapper/3: [name:spm&][SPM] suspend wake up by
R12_CCIF0_EVENT_B, timer_out = 5142432, r13 = 0x1b4c, debug_flag = 0x10030000 0x1, r12 =
0x100, r12_ext = 0x100, raw_sta = 0x100 0x0 0x0, idle_sta = 0x0, req_sta = 0x1000f8 0x10000
0x2002 0x2002 | 0x2002 0x380000 0x0 0x78 | 0x0 0x0 0xf5fcd800 0x7 | 0x5f800000 0x7c 0x3800
0x0 | 0x40, spm_src_req = 0x18f20000, cg_check_sta = 0x0, isr = 0x0, sw_rsv = 0x0 0x0 0x0
0x80000000 | 0x0 0xffffffff 0x0, raw_ext_sta = 0x848b155, wake_misc = 0x180030, sw_flag =
0x8a0014 0x50000, b_sw_flag = 0x8a0014 0x50000, clk_settle = 0x60fe, wlk_cntcv_l = 0x6d47dbc,
wlk_cntcv_h = 0xa, 26M_off_pct = 0, vcore_off_pct = 0
[ 3147.743642][T103000000000] swapper/3: [name:spm&][SPM] suspend warning:(OneShot)
System LPM is blocked by md
[ 3147.743642][T103000000000] swapper/3: [name:spm&][SPM] HWCG sta :[0] 0x0 0x0 0x0
[1] 0x0 0x0 0x0 [2] 0x0 0x0 0x0 [3] 0x0 0x0 0x0 [4] 0x0 0x0 0x0 [5] 0x0 0x0 0x0 .
[ 3147.743642][T103000000000] swapper/3: [name:spm&][SPM] PERI_CG sta :[0] 0x8 [1] 0x8
[2] 0x8 [3] 0x8 [4] 0x8 [5] 0x8 .
[ 3147.743642][T103000000000] swapper/3: [name:spm&][SPM] suspended for 156.934
seconds
[ 3147.743642][T103000000000] swapper/3: [name:spm&][lpm_suspend_s2idle_reflect:148] -
resume
```

若是没有打印出模块, 需要给出下面 spm 信息先定位是哪个模块然后再抓对应 owner

```
[ 147.324928][T102000000000] swapper/2: [name:spm&][SPM] suspend wake up by
R12_EINT_EVENT_B, timer_out = 10154156, r13 = 0x1200, debug_flag = 0x10030000 0x1, r12 =
0x40, r12_ext = 0x40, raw_sta = 0x40 0x0 0x0, idle_sta = 0x0, req_sta = 0x100000 0x0 0x0 0x0 |
0x0 0x380000 0x0 0x78 | 0x0 0x0 0x1fcd800 0x0 | 0x5f800000 0x0 0x0 0x0 | 0x40, spm_src_req =
0x18f20000, cg_check_sta = 0x0, isr = 0x0, sw_rsv = 0x0 0x0 0x0 0x80000000 | 0x0 0xffffffff 0x0,
raw_ext_sta = 0x84cb155, wake_misc = 0x180030, sw_flag = 0x8a0014 0x50000, b_sw_flag =
0x8a0014 0x50000, clk_settle = 0x60fe, wlk_cntcv_l = 0x68679ee3, wlk_cntcv_h = 0x1,
26M_off_pct = 0, vcore_off_pct = 0
[ 147.324928][T102000000000] swapper/2: [name:spm&][SPM] suspend warning:(OneShot)
System LPM is blocked by
[ 147.324928][T102000000000] swapper/2: [name:spm&][SPM] HWCG sta :[0] 0x0 0x0 0x0 [1]
0x0 0x0 0x0 [2] 0x0 0x0 0x0 [3] 0x0 0x0 0x0 [4] 0x0 0x0 0x0 [5] 0x0 0x0 0x0 .
[ 147.324928][T102000000000] swapper/2: [name:spm&][SPM] PERI_CG sta :[0] 0x8 [1] 0x8 [2]
0x8 [3] 0x8 [4] 0x8 [5] 0x8 .
```

C) 软件没有发现异常但是电流仍不达标

若出现下面字样, 说明软件休完了

```
swapper/3: [name:spm&][SPM] suspend wake up by R12_EINT_EVENT_B, timer_out = 19316402,
r13 = 0x1200, debug_flag = 0x1003f3ff 0x207e071, r12 = 0x40, r12_ext = 0x40, raw_sta = 0x40
0x0 0x0, idle_sta = 0x0, req_sta = 0x100000 0x0 0x0 0x0 | 0x0 0x380000 0x0 0x78 | 0x0 0x0
0x1fcd800 0x0 | 0x0 0x0 0x0 0x0 | 0x40, spm_src_req = 0x8f20000, cg_check_sta = 0x0, isr = 0x0,
sw_rsv = 0x0 0x0 0x0 0x80000000 | 0x126bd57 0xffffffff 0x0, raw_ext_sta = 0x84cb255, wake_misc
= 0x180030, sw_flag = 0x8a0014 0x50000, b_sw_flag = 0x8a0014 0x50000, clk_settle = 0x60fe,
wlk_cntcv_l = 0x94699ab0, wlk_cntcv_h = 0x2, 26M_off_pct = 99, vcore_off_pct = 99
```

若此时量测电流还是不达标, dump gpio 看 gpio 是否都有拉低

echo gpio_dump 1 > /proc/mtk_lpm/suspend/suspend_state

```
<5>[ 1714.940614][T10300002393] sleepmanager: PIN: (MODE) (DIR) (DOUT) (DIN) (DRIVE) (SMT) (IES) (PULL_EN) (PULL_SEL) (R1 R0)
<5>[ 1714.942155][T10300002393] sleepmanager: 000: 0001001110
<5>[ 1714.943132][T10300002393] sleepmanager: 001: 0111001110
<5>[ 1714.944065][T10300002393] sleepmanager: 002: 0000001110
```

1.4.6 唤醒问题

A) 点了唤醒没反应

下面命令用来看唤醒的 gpio 是否能被 SOS 识别

```
cat /proc/mtk_gpio/soc.pinctrl
```

B) rtc 定时未唤醒

1. 确定 rtc 是否有设置成功

```
行 2377: <5>[ 6859.793976][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc: set al time =
2025/09/22 10:25:07 (1)
```

```
行 2386: <6>[ 6859.840387][T10400057489] kworker/4:0: [mtk_rtc_set_alarm] from
vms
```

```
行 2389: <6>[ 6859.855519][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc:
[HWID 0x8530, MCLK 0x1, prot key 0x3] SEC write 7, latest 0
```

```
行 2392: <6>[ 6859.893871][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc:
[HWID 0x8530, MCLK 0x1, prot key 0x3] MIN write 25, latest 0
```

```
行 2393: <6>[ 6859.899981][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc:
[HWID 0x8530, MCLK 0x1, prot key 0x3] HOUR write 10, latest 0
```

```
行 2395: <6>[ 6859.914231][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc:
[HWID 0x8530, MCLK 0x1, prot key 0x3] DOM write 22, latest 1
```

```
行 2396: <6>[ 6859.920126][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc:
[HWID 0x8530, MCLK 0x1, prot key 0x3] MTH write 9, latest 1
```

```
行 2399: <6>[ 6859.930806][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc:
[HWID 0x8530, MCLK 0x1, prot key 0x3] YEAR write 57, latest 2
```

```
行 2402: <6>[ 6859.936343][T10400057489] kworker/4:0: mt6685-rtc mt6685-rtc: check rtc set
alarm
```

2. 确认 rtc 设定和休眠时间

先确认 rtc 是在哪段下的, 最好要三端 mobilelog

set al time 看 rtc 设置时间

```
<6>[ 322.398957][T10000000896] kworker/0:2: _mtk_rtc_check_alarm: Current time = 2025/10/25 03:05:24  
<5>[ 322.401220][T10000000896] kworker/0:2: mt6685-rtc mt6685-rtc: set al time = 2025/10/25 03:07:25 (1)  
<6>[ 322.402722][T10000000896] kworker/0:2: [mtk_rtc_set_alarm] from vms
```

看开始休眠时间

```
sleepmanager: [name:spm&] PM: suspend entry 2025-10-25 03:09:45.642960 UTC;android time 2025-10-25 03:09:45.642960
```

这个问题就是 rtc 设定的时候早于休眠时间导致没有唤醒, 属于正常情况

C)唤醒时间过长问题

yocto sos 唤醒

```
[ 2933.682725][ -T0] [name:spm&][lpm_suspend_s2idle_reflect:148] - resume  
[ 2935.177698][T171341] Tbox Resume vmid:1, param:1, flag:0
```

这中间有大量下面报错占时间

```
***[T13104]aic3x_write_register-ERR: i2c error 2== -6
```

tbox 唤醒

```
[ 2935.177698][T171341] Tbox Resume vmid:1, param:1, flag:0  
[ 2935.512013][T171362] android resume vmid:0
```

android 唤醒

```
[ 2870.155517][T30442] [tfa_time_sync_resume]time_to_sync:0x29c42999b0c  
[ 2870.214497][T30442] notify_android: resume  
[ 2870.945873][T30442] PM: suspend exit
```